EC302 — Digital System Design

Semester III • 3L:0T:0P • 3 Credits

Module 4 Exam Notes: VLSI Design Flow & VHDL Modeling

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1 The VLSI Design Flow and Methodologies **2**
1.1 The Gajski-Kuhn Y-Chart 2
1.2 Step-by-Step Modern Digital ASIC and FPGA Design Flow 2
2 VHDL Language Fundamentals and Structural Units **3**
2.1 Fundamental Design Units of VHDL 4
2.2 VHDL Port Modes 4
3 Data Types, Data Objects, and Operators **5**
3.1 Data Objects in VHDL 5
3.2 Standard and IEEE 1164 Data Types 5
3.3 VHDL Operators 6
4 VHDL Modeling Styles and Coding Methodologies **6**
4.1 Dataflow (Concurrent) Modeling 6
4.2 Behavioral (Sequential) Modeling 7
4.3 Structural Modeling 7
5 VHDL Code Library for Combinational Logic Circuits **7**
6 VHDL Code Library for Sequential Logic Circuits **10**

1. The VLSI Design Flow and Methodologies

Very Large Scale Integration (VLSI) design is the systematic transformation of an abstract architectural system specification into a physically verified, fabricated silicon integrated circuit containing millions to billions of transistors.

1.1 The Gajski-Kuhn Y-Chart

The Gajski-Kuhn Y-chart visualizes VLSI design complexity across three distinct **domains of representation** intersecting at concentric **levels of abstraction**:

- **Behavioral Domain (What it does):** Specifies functional algorithms, equations, input-output relationships, and state transitions without detailing physical hardware.
- **Structural Domain (How it is composed):** Defines hardware entities as interconnections of structural sub-blocks, modules, logic gates, and transistors.
- **Physical / Geometric Domain (How it is built):** Represents physical implementation as silicon geometric masks, transistor layouts, placement coordinates, and metallization routing tracks.
- **Levels of Abstraction:** Architectural / System level $\rightarrow$ Algorithmic level $\rightarrow$ Register-Transfer Level (RTL) $\rightarrow$ Logic / Gate level $\rightarrow$ Transistor / Circuit level $\rightarrow$ Mask layout level.

1.2 Step-by-Step Modern Digital ASIC and FPGA Design Flow

1. **System Specification & Architecture Planning:** Defines system throughput, clock frequency, word width, power budget, and interface protocols.
2. **Design Entry:** Entering the digital design into CAD tools:
   - *Schematic Capture:* Graphical drawing of interconnected logic gates. Intuitive for small designs; unmanageable for complex million-gate chips.
   - *State Diagram / FSM Entry:* Graphical specification of states, transitions, and outputs; automatically compiled into HDL code.
   - *Hardware Description Languages (HDLs):* Textual algorithmic descriptions using VHDL, Verilog, or SystemVerilog. Standard for modern complex VLSI.
3. **Functional (RTL) Simulation:** Verifies behavioral and logic correctness using an HDL testbench. Assumes ideal zero-delay or unit-delay gates to detect algorithmic bugs early.
4. **Logic Synthesis:** CAD tool (e.g., Synopsys Design Compiler, Vivado) translates abstract RTL code into an optimized, technology-specific **gate-level netlist** mapped to standard cells (AND, NAND, Flip-Flops) from a target semiconductor library, driven by designer constraints (target clock period, maximum silicon area, dynamic power).
5. **Gate-Level Netlist Simulation:** Simulates the synthesized gate netlist to confirm that synthesis introduced no unintended logic changes or state mismatches.

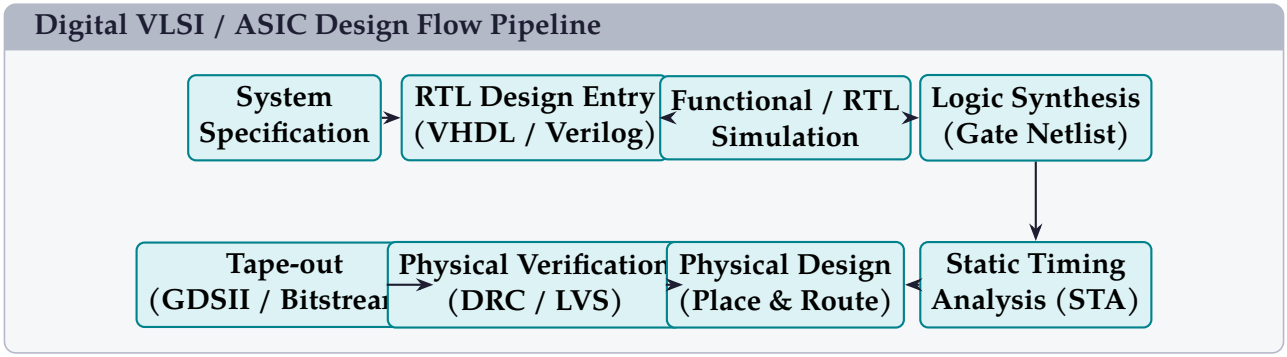
© Biraj

6. **Static Timing Analysis (STA):** Exhaustively evaluates path delays to guarantee that clock setup and hold time requirements are met for every register without dynamic vectors:

$$\text{Setup Time Constraint:} \quad T_{\text{clk}} \geq t_{cq} + t_{\text{comb(max)}} + t_{\text{setup}} - t_{\text{skew}} \tag{1}$$

$$\text{Hold Time Constraint:} \quad t_{cq} + t_{\text{comb(min)}} \geq t_{\text{hold}} + t_{\text{skew}} \tag{2}$$

7. **Physical Design (P&R - Place and Route):**
   - *Floorplanning:* Allocates chip area for major functional blocks, I/O pads, and power/ground grids (V_{DD}, V_{SS}).
   - *Placement:* Determines exact (x, y) coordinate locations for every standard cell.
   - *Clock Tree Synthesis (CTS):* Constructs balanced clock distribution networks (H-tree) to minimize clock skew (t_{skew}) and insertion delay.
   - *Routing:* Wires together all cell pins using multi-layer metallization.
8. **Parasitic Extraction and Post-Layout Timing Verification:** Extracts parasitic resistances (R) and capacitances (C) from actual layout geometries and back-annotates them into a Standard Delay Format (SDF) file for final timing sign-off.
9. **Physical Verification:**
   - *DRC (Design Rule Checking):* Verifies that geometric shapes satisfy foundry manufacturing rules (minimum metal width, spacing, enclosure).
   - *LVS (Layout Versus Schematic):* Extracts electrical transistor circuits from the physical layout and compares connectivity netlists 1-to-1 against the schematic.
10. **Tape-out:** Exporting final layout data in GDSII / OASIS stream format to the semiconductor foundry for photolithographic mask fabrication (or generating a configuration bitstream for an FPGA).



2. VHDL Language Fundamentals and Structural Units

VHDL (VHSIC Hardware Description Language) is an IEEE standard (IEEE Std 1076) language designed to model the behavior, dataflow, and structural composition of digital electronic systems across multiple levels of abstraction.

© Biraj

2.1 Fundamental Design Units of VHDL

A complete VHDL design entity is composed of primary design units:

1. **Library and Package Declarations:** Grants access to pre-compiled packages containing standardized data types, constants, and operators:

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;     -- Imports STD_LOGIC, STD_LOGIC_VECTOR
USE IEEE.NUMERIC_STD.ALL;        -- Imports SIGNED, UNSIGNED arithmetic
```

2. **Entity Declaration:** Defines the external hardware boundary and interface ports of the module (the "black box" view):

```
ENTITY entity_name IS
 GENERIC ( param_name : type := default_val ); -- Optional parameterized constants
  PORT (
    port_name : mode data_type;
    ...
  );
END entity_name;
```

3. **Architecture Body:** Defines the internal behavior, logic equations, or structural interconnection of components (the "inside" of the box). Multiple architectures can be written for a single entity (e.g., behavioral vs. structural):

```
ARCHITECTURE arch_name OF entity_name IS
  -- Declarative region: signals, constants, components, types
BEGIN
  -- Concurrent statements region: processes, component instantiations
END arch_name;
```

4. **Configuration Declaration:** Binds specific architecture bodies and component implementations to entity instances in hierarchical designs.

2.2 VHDL Port Modes

Port Mode	Directionality	Internal Architecture Usage
IN	**Input to module**	Read-only inside architecture. Cannot be assigned or driven.
OUT	**Output from module**	Write-only. Values can be assigned to it, but cannot be read internally.
INOUT	**Bidirectional**	Can be driven and read internally. Requires tri-state resolution.
BUFFER	**Output with internal feedback**	Drives output pin and allows internal readback. Has restrictive binding rules.

© Biraj

3. Data Types, Data Objects, and Operators

3.1 Data Objects in VHDL

VHDL supports four classes of data objects:

- **SIGNAL:** Represents physical interconnecting electrical wires or traces.
  - Declared in architecture declarative regions or packages; visible throughout the entire architecture.
  - Updated concurrently with a scheduled propagation delay (delta delay δ).
  - Assignment operator: `<=` (e.g., `Y <= A AND B;`).
- **VARIABLE:** Represents local computational scratchpad storage.
  - Declared strictly inside a `PROCESS`, `FUNCTION`, or `PROCEDURE`. Not visible outside.
  - Updated **instantaneously** without delay.
  - Assignment operator: `:=` (e.g., `v_count := v_count + 1;`).
- **CONSTANT:** Holds a permanent, unmodifiable value established during elaboration. Assignment operator: `:=` (e.g., `CONSTANT WIDTH : INTEGER := 8;`).
- **FILE:** Represents external host system files used for automated testbench verification and stimulus vectors.

Property	SIGNAL	VARIABLE
Hardware Meaning	Physical electrical wire / bus	Local memory / register variable
Declaration Scope	Architecture, package, block	Exclusively inside `PROCESS` or subprograms
Assignment Operator	`<=` (Non-blocking, scheduled)	`:=` (Blocking, instantaneous)
Update Timing	Updated at end of current simulation cycle	Updated immediately when evaluated
Synthesized Hardware	Interconnecting wires, latches, or flip-flops	Internal combinational nodes or registers

3.2 Standard and IEEE 1164 Data Types

- **Standard Types:** `BIT` (`'0'`, `'1'`), `BOOLEAN` (`TRUE`, `FALSE`), `INTEGER` (32-bit minimum range: -2^{31} to $2^{31}-1$), `CHARACTER`, `TIME` (e.g., `10 ns`).
- **IEEE Standard 1164 Multi-Valued Logic (`STD_LOGIC`):** Provides realistic modeling of electrical logic states across integrated circuit nodes:

© Biraj

Value	Electrical Meaning	Hardware Significance
`'U'`	**Uninitialized**	Default initial simulation state of unassigned signals
`'X'`	**Forcing Unknown**	Unresolvable contention / unknown logic conflict
`'0'`	**Forcing 0**	Strong logic LOW tied to ground (V_{SS})
`'1'`	**Forcing 1**	Strong logic HIGH tied to supply (V_{DD})
`'Z'`	**High Impedance**	Tri-state output; electrically disconnected
`'W'`	**Weak Unknown**	Weak signal contention
`'L'`	**Weak 0**	Pull-down resistor to ground
`'H'`	**Weak 1**	Pull-up resistor to supply
`'-'`	**Don't Care**	Wildcard used in synthesis minimization

3.3 VHDL Operators

- **Logical Operators:** `AND`, `OR`, `NAND`, `NOR`, `XOR`, `XNOR`, `NOT`.
- **Relational Operators:** `=` (equal), `/=` (not equal), `<`, `<=`, `>`, `>=`.
- **Arithmetic Operators:** `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), `MOD`, `REM`, `ABS`, `**` (exponentiation).
- **Concatenation Operator (`&`):** Combines scalars or vectors into wider vectors (e.g., `Y <= A & B;`).

4. VHDL Modeling Styles and Coding Methodologies

Digital hardware architectures can be expressed using three distinct modeling styles:

4.1 Dataflow (Concurrent) Modeling

Expresses hardware as continuous concurrent signal assignments that execute simultaneously whenever any right-hand-side operand changes:

- **Simple Concurrent Assignment:** `Y <= A AND B;`
- **Conditional Signal Assignment (`WHEN-ELSE`):**

```
Y <= A WHEN (Sel = "00") ELSE
     B WHEN (Sel = "01") ELSE
     C WHEN (Sel = "10") ELSE D;
```

- **Selected Signal Assignment (`WITH-SELECT`):**

© Biraj

```
WITH Sel SELECT
  Y <= A WHEN "00",
       B WHEN "01",
       C WHEN "10",
       D WHEN OTHERS;
```

4.2 Behavioral (Sequential) Modeling

Specifies the algorithmic behavior of the hardware within a **PROCESS** block. Statements inside a process execute sequentially from top to bottom, but the process block itself is evaluated concurrently relative to the rest of the architecture:

- **Sensitivity List:** A list of signals enclosed in parentheses following the `PROCESS` keyword. The process activates whenever any signal in its sensitivity list transitions:

```
PROCESS (clk, reset) -- Activated on transitions of clk or reset
BEGIN
  IF (reset = '1') THEN
    Q <= '0';
  ELSIF rising_edge(clk) THEN
    Q <= D;
  END IF;
END PROCESS;
```

- **Sequential Control Statements:** `IF-THEN-ELSIF-ELSE`, `CASE-WHEN`, `FOR-LOOP`, `WHILE-LOOP`.

Critical Synthesis Hazard: Inferred Latches

In combinational processes, if an `IF` statement lacks a complete `ELSE` clause, or a `CASE` statement omits the `WHEN OTHERS` branch or fails to assign every output signal across all possible input permutations, the synthesis tool must infer a **transparent latch** to preserve the previous output state. This creates asynchronous hazards, degrades timing closure, and must be prevented by assigning unambiguous default values at the top of every combinational process.

4.3 Structural Modeling

Describes a system hierarchically by declaring pre-designed sub-components and mapping their ports together like an electrical schematic:

1. Declare component interface in architecture declarative region.
2. Instantiate components in architecture body and connect signals using `PORT MAP`.

5. VHDL Code Library for Combinational Logic Circuits

© Biraj

1. Half Adder and Full Adder (Dataflow, Behavioral, Structural)

```
-- Half Adder (Dataflow)
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY half_adder IS
  PORT ( A, B : IN STD_LOGIC; Sum, Cout : OUT STD_LOGIC );
END half_adder;
ARCHITECTURE dataflow OF half_adder IS
BEGIN
  Sum  <= A XOR B;
  Cout <= A AND B;
END dataflow;

-- Full Adder (Structural using 2 Half Adders + OR Gate)
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY full_adder IS
  PORT ( A, B, Cin : IN STD_LOGIC; Sum, Cout : OUT STD_LOGIC );
END full_adder;
ARCHITECTURE structural OF full_adder IS
  COMPONENT half_adder
    PORT ( A, B : IN STD_LOGIC; Sum, Cout : OUT STD_LOGIC );
  END COMPONENT;
  SIGNAL s1, c1, c2 : STD_LOGIC;
BEGIN
  HA1: half_adder PORT MAP ( A => A, B => B, Sum => s1, Cout => c1 );
  HA2: half_adder PORT MAP ( A => s1, B => Cin, Sum => Sum, Cout => c2 );
  Cout <= c1 OR c2;
END structural;
```

2. 4-to-1 Multiplexer (Dataflow and Behavioral)

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY mux4to1 IS
  PORT (
    D   : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
    Sel : IN  STD_LOGIC_VECTOR(1 DOWNTO 0);
    Y   : OUT STD_LOGIC
  );
END mux4to1;

ARCHITECTURE behavioral OF mux4to1 IS
BEGIN
  PROCESS (D, Sel)
  BEGIN
    CASE Sel IS
      WHEN "00" => Y <= D(0);
```


© Biraj

```
      WHEN "01" => Y <= D(1);
      WHEN "10" => Y <= D(2);
      WHEN "11" => Y <= D(3);
      WHEN OTHERS => Y <= '0';
    END CASE;
  END PROCESS;
END behavioral;
```

3. 2-to-4 Binary Decoder with Active-Low Enable

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY decoder2to4 IS
  PORT (
    A     : IN  STD_LOGIC_VECTOR(1 DOWNTO 0);
    En_n  : IN  STD_LOGIC;                      -- Active-low enable
    Y     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)    -- Active-high outputs
  );
END decoder2to4;

ARCHITECTURE dataflow OF decoder2to4 IS
BEGIN
  Y <= "0001" WHEN (En_n = '0' AND A = "00") ELSE
       "0010" WHEN (En_n = '0' AND A = "01") ELSE
       "0100" WHEN (En_n = '0' AND A = "10") ELSE
       "1000" WHEN (En_n = '0' AND A = "11") ELSE
       "0000";
END dataflow;
```

4. 4-Bit Arithmetic Logic Unit (ALU)

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.NUMERIC_STD.ALL;

ENTITY alu_4bit IS
  PORT (
    A, B   : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
    Op     : IN  STD_LOGIC_VECTOR(2 DOWNTO 0);
    Result : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
    Cout   : OUT STD_LOGIC
  );
END alu_4bit;

ARCHITECTURE behavioral OF alu_4bit IS
BEGIN
```


© Biraj

```
  PROCESS (A, B, Op)
    VARIABLE temp : UNSIGNED(4 DOWNTO 0);
  BEGIN
    Cout <= '0';
    CASE Op IS
      WHEN "000" => -- ADD
        temp := ('0' & UNSIGNED(A)) + ('0' & UNSIGNED(B));
        Result <= STD_LOGIC_VECTOR(temp(3 DOWNTO 0));
        Cout   <= temp(4);
      WHEN "001" => -- SUBTRACT
        temp := ('0' & UNSIGNED(A)) - ('0' & UNSIGNED(B));
        Result <= STD_LOGIC_VECTOR(temp(3 DOWNTO 0));
      WHEN "010" => -- AND
        Result <= A AND B;
      WHEN "011" => -- OR
        Result <= A OR B;
      WHEN "100" => -- XOR
        Result <= A XOR B;
      WHEN "101" => -- NOT A
        Result <= NOT A;
      WHEN OTHERS =>
        Result <= "0000";
    END CASE;
  END PROCESS;
END behavioral;
```

6. VHDL Code Library for Sequential Logic Circuits

1. D Flip-Flop with Asynchronous Active-Low Reset

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY d_flip_flop IS
  PORT (
    clk     : IN  STD_LOGIC;
    rst_n   : IN  STD_LOGIC; -- Asynchronous active-low reset
    d       : IN  STD_LOGIC;
    q, q_bar: OUT STD_LOGIC
  );
END d_flip_flop;

ARCHITECTURE behavioral OF d_flip_flop IS
  SIGNAL q_reg : STD_LOGIC;
BEGIN
  PROCESS (clk, rst_n)
  BEGIN
```


© Biraj

```
    IF (rst_n = '0') THEN
      q_reg <= '0';
    ELSIF rising_edge(clk) THEN
      q_reg <= d;
    END IF;
  END PROCESS;
  q     <= q_reg;
  q_bar <= NOT q_reg;
END behavioral;
```

2. 4-Bit Synchronous Up-Counter with Parallel Load

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.NUMERIC_STD.ALL;

ENTITY counter_4bit IS
  PORT (
    clk    : IN  STD_LOGIC;
   rst_n  : IN  STD_LOGIC;                -- Asynchronous active-low reset
    load   : IN  STD_LOGIC;                    -- Synchronous load enable
    en     : IN  STD_LOGIC;                      -- Count enable
    d_in   : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
    q_out  : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
  );
END counter_4bit;

ARCHITECTURE behavioral OF counter_4bit IS
  SIGNAL count : UNSIGNED(3 DOWNTO 0);
BEGIN
  PROCESS (clk, rst_n)
  BEGIN
    IF (rst_n = '0') THEN
      count <= (OTHERS => '0');
    ELSIF rising_edge(clk) THEN
      IF (load = '1') THEN
        count <= UNSIGNED(d_in);
      ELSIF (en = '1') THEN
        count <= count + 1;
      END IF;
    END IF;
  END PROCESS;
  q_out <= STD_LOGIC_VECTOR(count);
END behavioral;
```


© Biraj

3. 4-Bit Universal Shift Register

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY universal_shift_reg IS
  PORT (
    clk    : IN  STD_LOGIC;
    rst_n  : IN  STD_LOGIC;
   mode   : IN  STD_LOGIC_VECTOR(1 DOWNTO 0); -- 00: Hold, 01: Right, 10: Left, 11: Load
    sin_r  : IN  STD_LOGIC;                  -- Serial input for shift-right
    sin_l  : IN  STD_LOGIC;                   -- Serial input for shift-left
    pin    : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);  -- Parallel data in
    pout   : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)   -- Parallel data out
  );
END universal_shift_reg;

ARCHITECTURE behavioral OF universal_shift_reg IS
  SIGNAL r_reg : STD_LOGIC_VECTOR(3 DOWNTO 0);
BEGIN
  PROCESS (clk, rst_n)
  BEGIN
    IF (rst_n = '0') THEN
      r_reg <= (OTHERS => '0');
    ELSIF rising_edge(clk) THEN
      CASE mode IS
        WHEN "00" => r_reg <= r_reg;                    -- Hold
        WHEN "01" => r_reg <= sin_r & r_reg(3 DOWNTO 1); -- Shift Right
        WHEN "10" => r_reg <= r_reg(2 DOWNTO 0) & sin_l; -- Shift Left
        WHEN "11" => r_reg <= pin;                      -- Parallel Load
        WHEN OTHERS => NULL;
      END CASE;
    END IF;
  END PROCESS;
  pout <= r_reg;
END behavioral;
```

4. FSM Sequence Detector for Pattern ”1011” (Overlapping Mealy Machine)

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY seq_detector_1011 IS
  PORT (
    clk   : IN  STD_LOGIC;
    rst_n : IN  STD_LOGIC;
    din   : IN  STD_LOGIC;
    dout  : OUT STD_LOGIC
  );
END seq_detector_1011;
```


© Biraj

```
ARCHITECTURE fsm_arch OF seq_detector_1011 IS
  TYPE state_type IS (S0, S1, S10, S101);
  SIGNAL state_reg, next_state : state_type;
BEGIN
  -- Process 1: State Register (Synchronous)
  PROCESS (clk, rst_n)
  BEGIN
    IF (rst_n = '0') THEN
      state_reg <= S0;
    ELSIF rising_edge(clk) THEN
      state_reg <= next_state;
    END IF;
  END PROCESS;

  -- Process 2: Next State and Output Logic (Combinational)
  PROCESS (state_reg, din)
  BEGIN
    dout <= '0';
    next_state <= state_reg;
    CASE state_reg IS
      WHEN S0 =>
       IF (din = '1') THEN next_state <= S1; ELSE next_state <= S0; END IF;
      WHEN S1 =>
       IF (din = '0') THEN next_state <= S10; ELSE next_state <= S1; END IF;
      WHEN S10 =>
      IF (din = '1') THEN next_state <= S101; ELSE next_state <= S0; END IF;
      WHEN S101 =>
        IF (din = '1') THEN
          dout <= '1'; -- Pattern "1011" detected!
          next_state <= S1; -- Overlapping: prefix "1" retained
        ELSE
          next_state <= S10;
        END IF;
    END CASE;
  END PROCESS;
END fsm_arch;
```


© Biraj

Quick Revision — Module 4

VLSI Design Flow & VHDL Hardware Modeling | EC302

Key Timing Constraints and VHDL Skeletons at a Glance

Setup Time Constraint	$T_{\text{clk}} \geq t_{cq} + t_{\text{comb(max)}} + t_{\text{setup}} - t_{\text{skew}}$
Maximum Clock Frequency	$f_{\text{max}} = \frac{1}{T_{\text{clk(min)}}}$
Hold Time Constraint	$t_{cq} + t_{\text{comb(min)}} \geq t_{\text{hold}} + t_{\text{skew}}$ (independent of T_{clk})
Dynamic Power Dissipation	$P_{\text{dyn}} = \alpha \cdot C_L \cdot V_{DD}^2 \cdot f$
Entity Declaration Skeleton	`ENTITY name IS PORT (...); END name;`
Architecture Body Skeleton	`ARCHITECTURE arch OF name IS ... BEGIN ... END arch;`
Process Trigger Mechanism	`PROCESS (clk, reset) BEGIN ... END PROCESS;`

One-Line Definitions

- **Logic Synthesis:** The automated translation of abstract RTL code into an optimized gate netlist mapped to standard library cells.
- **Static Timing Analysis (STA):** A vector-independent CAD technique verifying setup and hold timing across all paths.
- **Clock Skew:** The difference in clock arrival times at two distinct registers caused by routing differences.
- **VHDL Entity:** The primary design unit defining external hardware interface ports and generic parameters.
- **VHDL Architecture:** The design unit describing internal hardware dataflow, algorithmic behavior, or structural netlist.
- **SIGNAL in VHDL:** A hardware object representing physical interconnecting wires, scheduled with delta delay.
- **VARIABLE in VHDL:** A local computational object evaluated sequentially and updated instantaneously inside a process.
- **Inferred Latch:** An unintended transparent latch generated by synthesis due to incomplete conditional assignments.
- **Sensitivity List:** The list of trigger signals in a process header; the process evaluates when any listed signal changes.
- **DRC (Design Rule Check):** CAD verification ensuring physical layout geometries comply with foundry manufacturing rules.

© Biraj

Frequently Asked / Viva Questions

Q1. What is the difference between a SIGNAL and a VARIABLE in VHDL?

A `SIGNAL` models physical electrical wires, is declared in the architecture declarative section, and updates with a scheduling delay (<=). A `VARIABLE` models local scratchpad data, is declared strictly inside a process/subprogram, and updates instantaneously (:=).

Q2. What causes an unintended transparent latch to be inferred during synthesis?

Incomplete `IF` statements (missing `ELSE`) or incomplete `CASE` statements (missing `WHEN OTHERS`) in combinational processes force synthesis tools to generate latches to preserve previous values.

Q3. Why is Static Timing Analysis (STA) preferred over dynamic timing simulation?

STA exhaustively checks setup and hold constraints across every single timing path in the chip without requiring test stimulus vectors, achieving 100% path coverage exponentially faster.

Q4. What is the physical meaning of the 9 values in IEEE STD_LOGIC?

It provides realistic simulation of digital states: `'U'` (uninitialized), `'X'` (forcing unknown), `'0'` (strong 0), `'1'` (strong 1), `'Z'` (high impedance), `'W'` (weak unknown), `'L'` (weak 0 / pull-down), `'H'` (weak 1 / pull-up), and `'-'` (don't care).

Q5. What is the difference between synchronous and asynchronous resets in VHDL?

An asynchronous reset appears in the process sensitivity list and executes immediately regardless of clock. A synchronous reset does not appear in the sensitivity list and evaluates strictly on the clock edge.

Q6. What is the Gajski-Kuhn Y-Chart?

A geometric framework visualizing VLSI design across three representations (Behavioral, Structural, Physical) intersecting through multiple levels of abstraction.

Q7. What is the purpose of Layout Versus Schematic (LVS) verification?

LVS extracts device connectivity directly from fabricated geometric mask layouts and verifies that it electrically matches the synthesized circuit schematic 1-to-1.

Q8. Why cannot signals assigned inside a process be read immediately within the same cycle?

Because signal assignments inside a process are non-blocking and scheduled for update only after the process suspends (after delta delay δ).

Q9. What is the difference between functional simulation and timing simulation?

Functional simulation verifies RTL logic algorithms with ideal zero delay. Timing simulation annotates post-layout parasitic delays (via an SDF file) to verify real hardware delays.

© Biraj

Frequently Asked / Viva Questions (contd.)

Q10. What is the role of Clock Tree Synthesis (CTS) in physical design?

CTS creates a balanced distribution network (such as an H-tree) to ensure clock edges arrive at all flip-flops simultaneously, minimizing clock skew.

Q11. Explain the difference between positional association and named association in port mapping.

Positional association connects signals based on strict order in the entity port list. Named association links signals explicitly by name (`port_name => signal_name`), preventing wiring errors.

Q12. What is the difference between synthesizable and non-synthesizable VHDL constructs?

Synthesizable constructs map directly onto physical standard cells (e.g., gates, flip-flops). Non-synthesizable constructs (e.g., `wait for 10 ns`, file I/O) are used solely in simulation testbenches.

Summary Matrix: Comparison of VHDL Modeling Styles

Feature	Dataflow Modeling	Behavioral Modeling	Structural Modeling
Primary Construct	Concurrent signal assignments	`PROCESS` with sequential statements	`COMPONENT` and `PORT MAP`
Execution Semantics	Concurrent (Event-driven)	Sequential inside process	Concurrent interconnection
Hardware Analogy	Boolean logic gates and data flow	Algorithmic behavior, state machines	Schematic netlist wiring
Typical Commands	`<=`, `when-else`, `with-select`	`if-then-else`, `case-when`, loops	`COMPONENT`, `PORT MAP`
Best Suited For	Small combinational modules, muxes	Complex controllers, counters, FSMs	Top-level integration, hierarchies

© Biraj